

The Trust Account Proof Layer, Drawn

This is the deepest document in the set. It is written for two readers: the partner who will build it, and the security reviewer at a customer who has to sign it off. Page two sets out each new word before you meet it.

9 LAYERS	4 THINGS THAT CANNOT BE ADDED LATER	2 DATES ON EACH RECORD, NOT ONE	0 WAYS TO CHANGE A STORED RECORD
-------------	---	---------------------------------------	--

Confidential. Prepared for a named reader. Please do not pass it on. Every figure is Canadian unless it says otherwise beside the figure. Figures drawn from the financial model of 5 August 2026, base case.

1 The duty behind the software, and the two readers of this pack

Why the software exists, in five steps
This is the duty a Canadian payment firm has carried since 8 September 2025.

1

MONEY COMES IN
A customer's pay, deposit or refund lands with the firm.

→

2

NOT THE FIRM'S
It sits in its own account. The firm looks after it and never owns it.

→

3

PROVE IT DAILY
What the firm says it holds, against what the bank says is there. Any gap is flagged.

→

4

KEEP THE PROOF
The result is dated, signed and kept. Nothing can change it later.

→

SHOW IT LATER
An examiner asks about one day, a year and a half after that day.

We never touch the money. We are not a bank and we hold no customer funds. What we sell is the proof the law asks to see.

A payment firm has held this duty since 8 September 2025. The five steps above are the whole of it. Everything in the rest of this pack is a choice about how to make step four and step five hold up in front of somebody who has every reason to doubt us.

4orm Finance · Confidential · Not for distribution

Architecture Pack · 5 August 2026 · Page 1 of 21

2 Words used in this document, and what each one means

These are the only special words in this paper, and nothing later uses one that is not here. They are set out in the order you meet them.

Trust account	A bank account holding money that belongs to somebody else. A lawyer holding your house deposit is the everyday example.
Payment company	A business that holds or moves money for other people without being a bank. Payroll firms, money transfer apps and gift card companies. These are our first customers.
Safeguarding	Keeping customer money apart from the company's own money, and being able to prove it.
Reconciliation	Checking two lists against each other. What a company says it holds for customers, against what the bank says is in the account.
Registration	Government permission to operate. A company applies and the regulator decides. Some are turned down.
Regulator	The government body that writes the rules and checks that a company follows them. Ours is the Bank of Canada.
Examiner	The person from the regulator who asks to see the records, usually about a date in the past.
Independent review	An outside firm checking the records. The first one falls due in September 2028 and it looks back three years.
Penalty ceiling	The largest fine a regulator can hand out for one breach. Here it is one million dollars for a serious one and ten million for a very serious one.
Recurring revenue	Money that comes in every year without having to sell it again. It is what a software company is valued on.
Pre-seed and seed	The first and second rounds of money a company raises. The first builds the product. The second grows it.
SAFE	A way of putting money in now and setting the price later, at the next priced round. It avoids arguing about the value of a company with no revenue yet.
Evidence store	The place the daily proof is kept. Records go in, get dated, and are never changed.

This pack in four parts, read in this order

1

THE MAP, AND NINE LAYERS

What comes in, what we run, who gets the proof

2

WHAT A RECORD IS

Two dates, eight fields, one chain

3

HOW A DAY RUNS

Seven steps, and every failure path

4

WHAT IT COSTS

Who builds each layer, and when

Every engineering word beyond this list is spelled out at the foot of the page that uses it.

3 Three things proof software does that ordinary software does not

Ordinary business software is judged by what it shows a user today. This one is judged by a stranger, about a day that has already gone, under a rule that may since have changed. Those three differences set every choice in the rest of this pack.

1 Remember, rather than store

Ordinary software keeps what is true now and writes over the rest. This one keeps each state it has held, with who put it there and when, because the question it gets asked is always about a day that has already gone.

2 Know which rule applied

Ordinary software puts today's rules in today's code. This one holds the rules as dated content, so a duty done in March 2026 is shown under the rule as it stood in March 2026.

3 Be believed by a stranger

Ordinary software is trusted because the customer trusts it. This one has to be trusted by an examiner and a reviewer who have no tie to us and every reason to doubt.

4 System map. What comes in, what we run, who gets the proof

What comes in four, all the firm's own

BANK AND TRUST

Daily statements, balances, movements

THE FIRM'S LEDGER

Each customer, and how much is held

CARD FIRM

Payout and float files

THE FIRM'S PEOPLE

Sign offs, approvals, incident detail

What we run seven parts, nine layers

4orm · the proof layer

Ingest, and names matched up



Rule store, rules held as content



The core: checks, cases, filings



Evidence store, added not changed



Two dates on every record



Filing packs, one per rule maker



Assistant, finds and drafts only

Who gets the proof three of four are strangers

BANK OF CANADA

Yearly report, incident and change notices

OUTSIDE REVIEWER

A proof export, taken without a follow up

THE EXAMINER

An indexed pack, built when asked

PROVINCIAL COUNCILS

Monthly trust check, and what is held

Look at the right hand column. Three of those four readers have no contract with us and no reason to be kind. Watching began on 8 September 2025, about three hundred firms are on the list and more than fifteen hundred have filed, and each one owes an outside review of its plan by 8 September 2028. **One build, three names.** Safeguard does the daily money check. Comply carries the plan, the tests, the notices and the 31 March report. Trust runs the same check monthly for a provincial council. One core sits under all three, and it does not know what a payment firm is.

WHAT EACH PART MEANS

Ingest. The part that takes files in from banks and other systems and checks them.

Evidence store. Where finished records are kept. Written once, never altered.

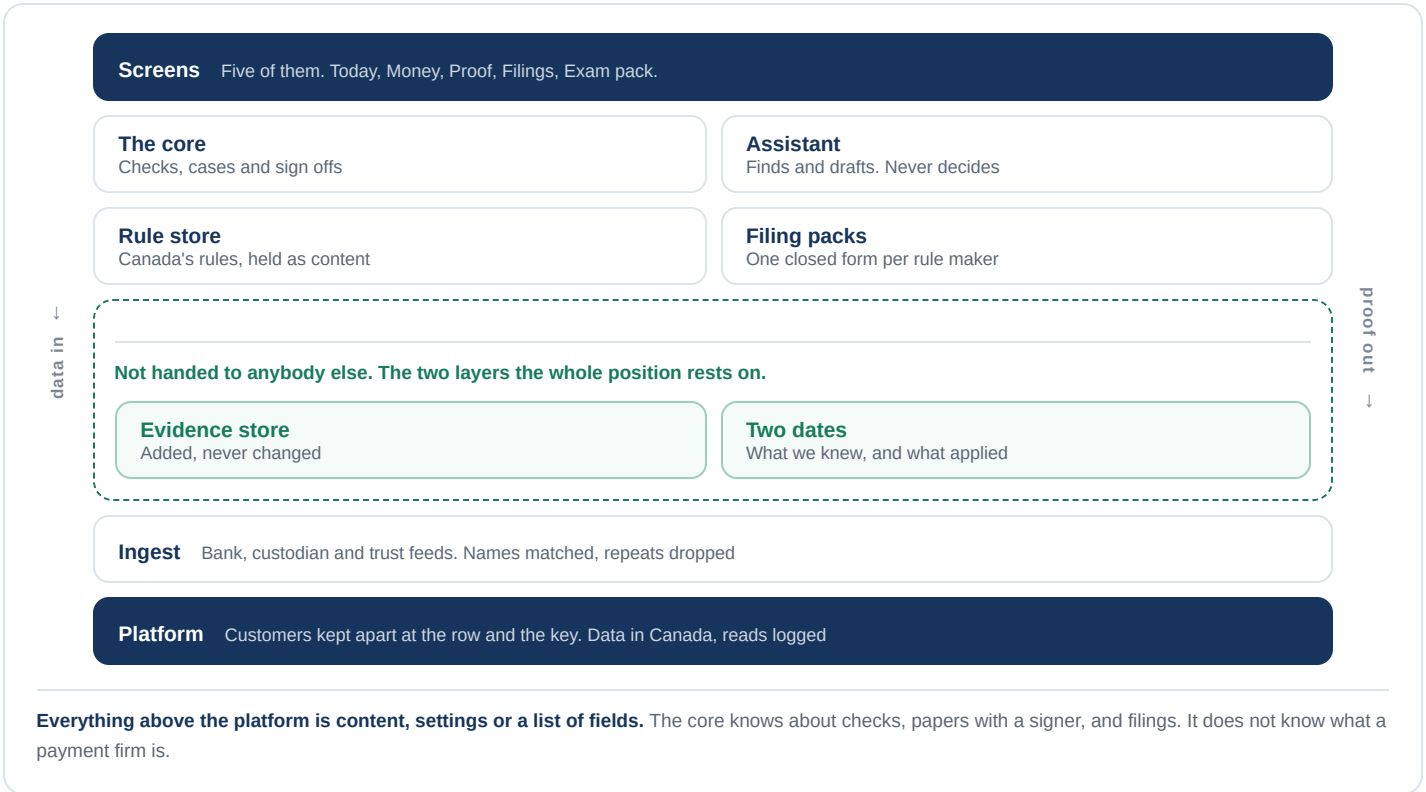
The core. The part that does the work: the compare, the sums and the test.

Screens. The five pages a person looks at. They ask for records and never make them. Engineers would call this the surface.

Rule store. Canada's rules held as dated content, so a rule change is a piece of writing rather than new code. Engineers would call it an obligation library.

Filing pack. The part that turns our records into the exact form one rule maker accepts, and keeps the receipt.

5 Nine layers



EVERY LAYER NAME ON THIS DRAWING, AND WHAT EACH ONE MEANS

Layer. One part of the build with a job of its own, which the parts above it can use without knowing how it works.

The core. The one build that does the work: the compare, the sums and the test.

Platform. The bottom layer. Where the data sits, who may reach it, and how it is brought back after a failure.

Rule store. Canada's rules held as dated content, so a rule change is a piece of writing rather than new code.

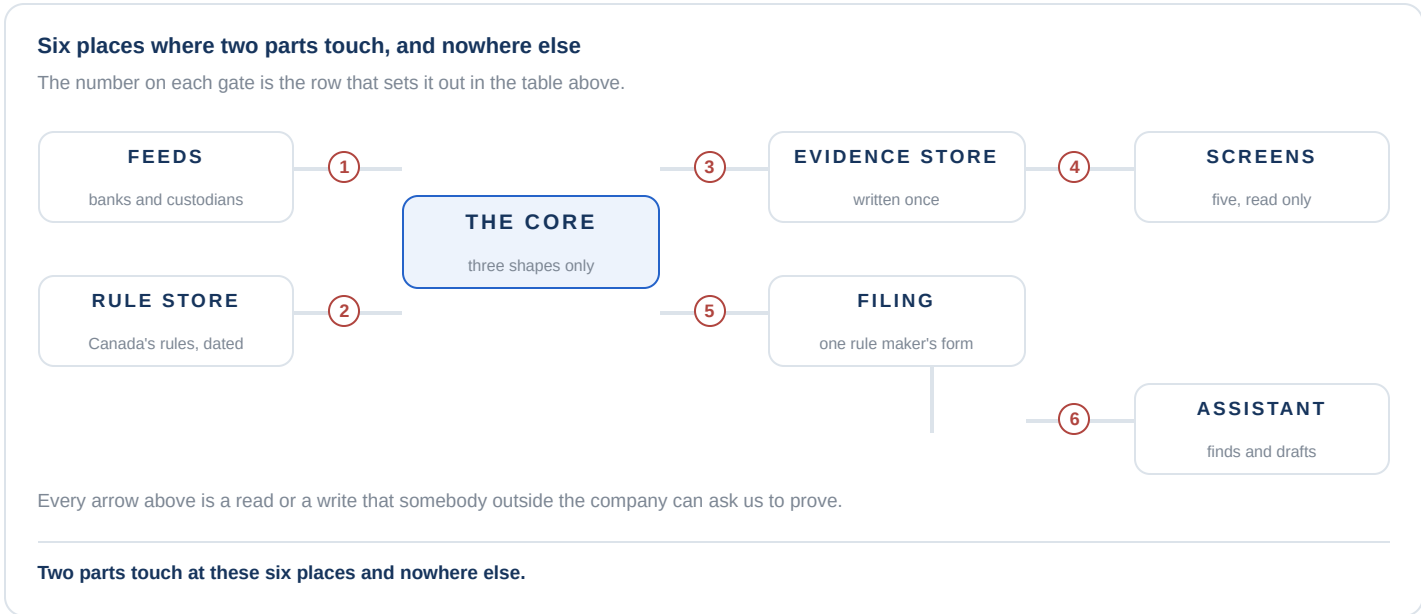
Screens. The five pages a person looks at. They ask for records and never make one.

Assistant. The part that searches and drafts inside one customer. It never writes proof.

6 Boundaries, and what must never cross them

BOUNDARY	WHAT MAY CROSS IT	WHAT MUST NEVER CROSS IT
Feeds into the core	A checked record with a source and the time it arrived.	A value quietly patched up. If a field will not read, the record fails rather than defaults.
Rule store into the core	A duty, the dates it applies to, and which version was in force on the day being worked.	A rule written as code. If a duty cannot be written as content, the core is short a shape.
Core into the evidence store	A whole result with its inputs, its rule version and its signer, written once.	Half a write, a change, or a pointer to something that can still move.
Evidence store into the screens	A read, logged as an event, scoped to one customer.	A write of any kind. The screens never make a record, they only ask for one.
Core into a filing	A checked payload matching the form that rule maker published for that year.	A made up field carrying something the form does not accept.
Assistant into anything	A search inside one customer, with each answer tied to a record id.	A made up value entering the record. The assistant drafts for a person. It never writes proof.

Six boundaries, and the right hand column matters more than the left. This design is set by what it refuses, because the refusals are the part a reviewer can test.



WHAT EACH TERM ON THIS PAGE MEANS		
Boundary. The line between two parts, and the only place they are allowed to touch.	Payload. The finished set of figures sent to a rule maker, in that rule maker's own form.	Record id. The label that points at one record and one record only.
Scoped. Cut down to one customer, one date range or one duty before anything is returned.	Defaults. Fills a blank with a guess. Not allowed here, at any boundary, for any reason.	Shape. One of the three things the core knows how to do: a check, a paper, a filing.

7 Four things ruled out, on purpose and for good

A design is easier to trust when it is written down as a list of refusals. Each line on the left is a thing this system will not do, in any release, for any customer. Each line on the right is what it does instead.

Never ships

- A way to change or delete a stored record, in any setting, for any reason, support included.
- A script that rewrites old rows. Changes to the shape of the data add. They never revise.
- A fix that wipes the record it replaces. Fixes sit beside the old record, which stays readable.
- An admin override, however carefully guarded. One exists or it does not, and one is enough.

Ships instead

- Records filed by content and locked to each other, so tampering shows rather than is merely frowned on.
- A replaces link held as data, so the story of a fix is itself part of the record.
- Month sealing, so a closed month is provably closed and a late write to it is in plain sight.
- A log of who read what on every read, including reads by us and reads by the founder.

8 Three ways these get dropped, in order of how likely they are

1 A demo date

The screens are seen and the store is not. When a date moves, the work nobody can see is what gets traded, and nobody in the room feels they have lost anything.

2 A support call

A customer has a wrong value and wants it fixed rather than replaced. One admin change, granted once, under pressure, ends the add only claim for good and in silence.

3 A sensible engineer

Dating each rule version looks like too much to somebody who has never sat in an exam. The case for it is a rule case rather than a build case, which is exactly why it is written down here.

9 Two questions an examiner asks that almost no system can answer

1 What did you know on the day?

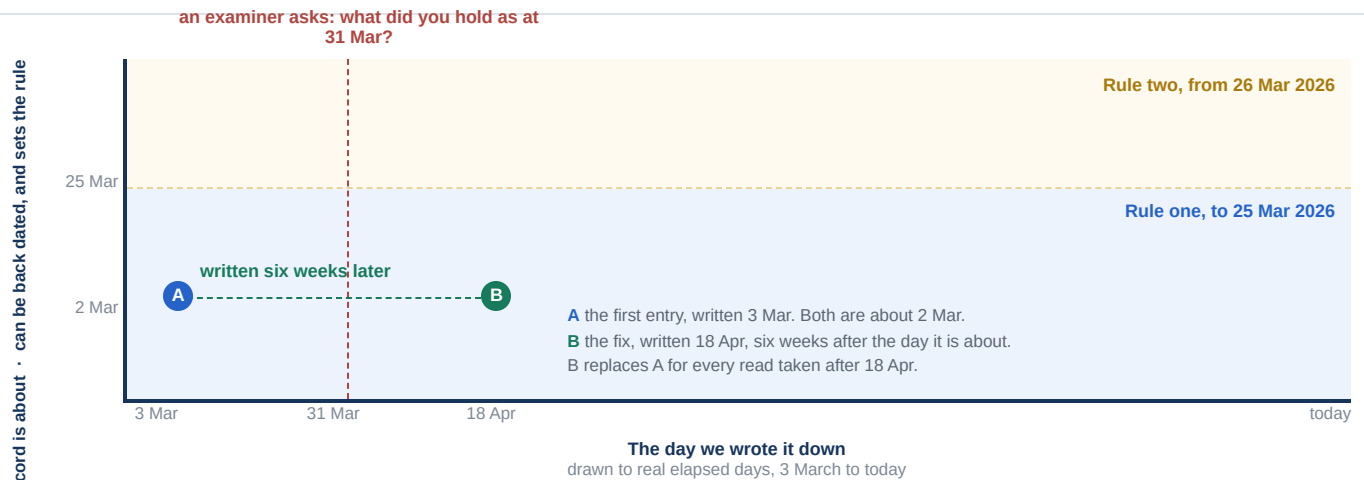
An examiner in November 2027 is not asking what the balance is now. She is asking what the record said on 14 March 2026, before the fix and before the person who typed it left. A system that keeps only what is true now can answer with today's truth and nothing else.

2 What did the rule say on the day?

The second date, and the one everybody forgets. A duty done in March 2026 was done under the rule as it stood then. The yearly form changes each year and the model list duty starts on 1 May 2027. A record that cannot name the version invites a second letter.

Both questions are about a day that has already gone. A system that keeps only what is true now has one answer to each of them, and it is the wrong one. That is the whole case for the two dates set out on the next page.

10 Two dates on every record, and the price of holding one



WITH ONE DATE

The system answers with today's version of the truth. It cannot say what it held on 31 March, or which rule it was using.

WITH TWO DATES

Read the plate up the red line and it hands back A, under rule one, with the signer and the time. The later fix does not move that answer.

The plane is the point. One line answers where a value stands now. A plane answers what we held on a given day, under the rule in force that day, with the signer attached. Each question this business gets asked is one point on that plane. Engineers call this a bitemporal model, which just means two dates.

EVERY ENGINEERING WORD ON THIS DRAWING, AND WHAT EACH ONE MEANS

The day it is about. The day the record covers. It can be back dated, and often is. Engineers call this valid time.

Back dated. A record about a day earlier than the day it was written.

The day we wrote it. The day we learned it and wrote it down. It only ever moves forward. Engineers call this recorded time.

Replaces. A later record that stands in for an earlier one. The earlier one stays, and stays readable.

Two dates. Holding both, so any past day can be asked about as it stood. Engineers call this bitemporal.

Rule version. Which wording of a duty was in force on the day being worked.

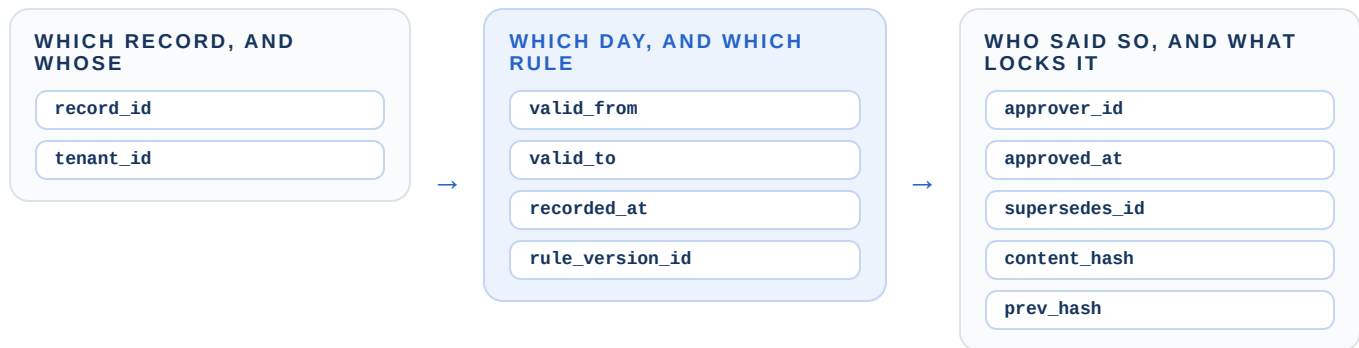
There is no version of this system that adds the second date in year two. The rule versions that applied to past choices were never kept, so they cannot be worked out from anything, at any price. A week of work at the start and a year of work later are not the same thing at two prices. They are a thing done and a thing that can no longer be done at all.

11 The record shape, agreed before a line is written

FIELD	TYPE	WHY IT EXISTS
record_id	uuid v7	Sorts in the order records were made, so that order can be got back without a second count.
tenant_id	uuid	Which customer the row belongs to. On every row, and checked below the query, not inside it.
valid_from, valid_to	date, date	The span the record is about. Can be back dated, and often is.
recorded_at	timestampz	When we learned it. Moves forward only, and is never revised.
rule_version_id	uuid	Which version of which duty was in force for the span being worked.
supersedes_id	uuid, null	Points at the record this one replaces. The replaced record stays and stays readable.
approver_id, approved_at	uuid, timestampz	A sign off is only proof if a person with the right to sign gave it, at a time.
content_hash, prev_hash	bytea, bytea	Its own fingerprint, and the fingerprint of the record before it. That pair is the chain.

Eight fields, and all eight carry weight. The pair that carries the most is valid_from and recorded_at together, because holding both is what turns a database into a record.

Eight fields, grouped by the question each one answers



Take one field away and a question somebody outside the company asks goes unanswered.

The middle group is the one ordinary software leaves out. It is also the group an examiner opens with, because both of its dates are about a day that has gone.

WHAT EACH TERM ON THIS PAGE MEANS

uuid. A long label that is unique everywhere, so two records can never clash.

Supersede. Replace an earlier record while leaving that earlier record in place and readable.

timestampz. A date and time held with its time zone, so the moment is never in doubt.

Tenant. One customer's own walled space inside shared software.

bytea. A block of raw data. Here it holds the fingerprint of a record.

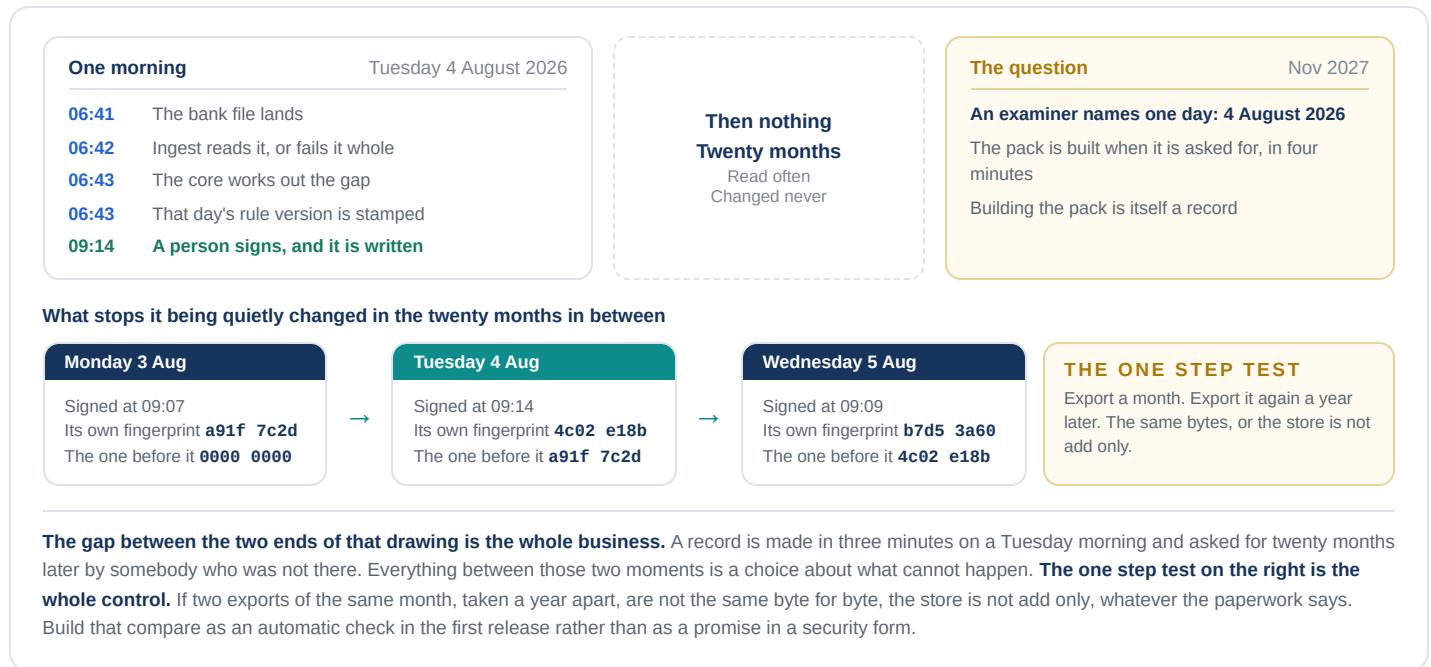
Null. Empty on purpose. Here it means this record replaces nothing.

Hash. That fingerprint. Change one letter and the fingerprint changes.

Field. One named box on a record, like a column on a spreadsheet.

12 The whole life of one record, and the chain that holds it still

One record, from the file that makes it to the examiner who asks for it. Time runs left to right. Nothing is added to the record after 09:14 on the morning it was made.



EVERY ENGINEERING WORD ON THIS DRAWING, AND WHAT EACH ONE MEANS

Ingest. The part that takes files in from banks and other systems and checks them.

Chain. Each record carries the fingerprint of the one before it, so altering one shows up in plain sight. Engineers call this a hash chain.

Rule version. Which version of which duty was in force on the day being worked.

Add only. New records can go in. Nothing already written can be changed or removed. Engineers call this append only.

Fingerprint. A short value worked out from the content of a record. Change one letter and it changes. Engineers call it a hash.

Export. A copy of the records, handed over in a form the reader can open without us.

A record is made in three minutes and asked for twenty months later.

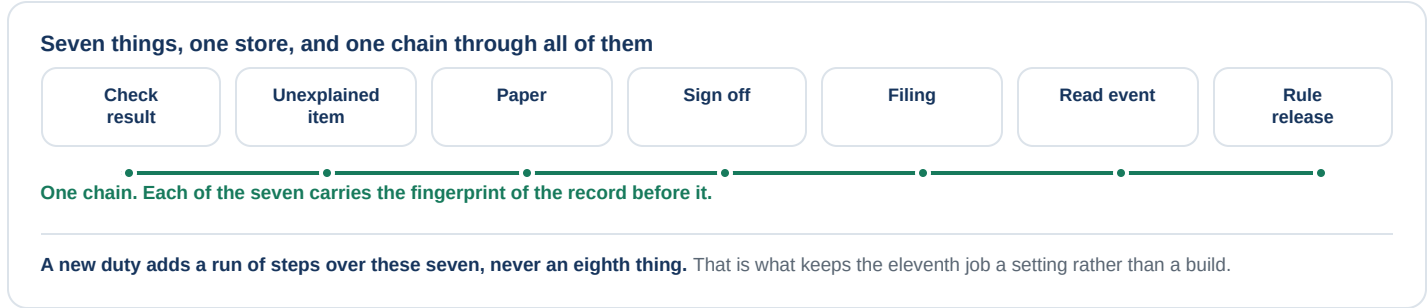
Everything in this pack is a choice about what may happen in the months in between. The honest answer, nearly every time, is nothing at all.

13

The seven things in the evidence store, and what each one carries

THING	MADE BY	CARRIES	SEALED
Check result	The core, alone	Both totals, the gap, the shortfall answer, the list of unexplained items and the rule version.	On sign off
Unexplained item	The core	The payments behind it, the likely cause, and the answer when one is written down.	With its parent
Paper	A person	The written thing, its version, its owner, its signer, and the duties it answers.	On sign off
Sign off	A person with the right to sign	Who, when, on what basis, and under whose authority.	At once
Filing	The core, then a person	The payload as sent, the form version, the receipt, and proof of the time it was sent.	On receipt
Read event	The platform	Who read what, when, from where, and under which ticket. Reads by us included.	At once
Rule release	The person who owns the rules	The duty, the dates it applies to, what changed, and the signature.	On release

Seven things, one store, one chain. A job is a run of steps over these seven rather than a new table, which is why the eleventh job needs no change to the shape of the data.



WHAT EACH TERM ON THIS PAGE MEANS

Add only. New records can go in. Nothing already written can be changed or removed. Engineers call this append only.

Filed by content. A record is filed under a fingerprint taken from its own content rather than under a name somebody chose.

Sealed. Closed and fixed at a stated moment, after which a late arrival shows up as late.

Who read what. The unbroken written history of who held a record and who read it, kept on the same terms as the record itself.

Chain. Each record carries the fingerprint of the one before it, so removing or altering one breaks the chain in plain sight. Engineers call this a hash chain.

Job. One whole duty the system runs end to end. Engineers call this a workload.

14 Four things that have to be right in the first release

1 Add only, in the store itself

- No way to change and no way to delete anywhere in the code, blocked by the store itself.
- Records filed by content, each locked to the one before it.
- Fixes replace and never edit. The replaced record stays and stays readable.
- Tested in one step: two exports of the same month, a year apart, compared byte for byte.

2 Two dates, each with a from and a to

- What the record said and when it said it, which is the system date.
- What the rule said and when it said it, which is the rule date and the one usually missing.
- Any choice can be run again exactly as it stood on any day, under that day's rule.
- Without it, a question about March 2026 gets answered with today's truth.

3 Rules as content, not code

- The rule store holds Canada's rules as data, with versions and the dates each one applied.
- Duty to proof links live in the same content, so a rule change re-points the proof by itself.
- A rule change is a signed release. Never a builder, never a code release.

4 Customers apart, data in Canada

- Kept apart at the row and at the key, not inside the query. A bad query is a breach.
- Data held in Canada, written into the contract, because a watched buyer asks in the first call.
- A read of a record is itself a logged event, including a read by us.

15 The cost of each of the four now, against the cost of adding it later

THING	COST NOW	WHAT ADDING IT LATER MEANS	TEST
Add only store	Days	Each record written before the change has no chain behind it, so the trail is provable only from the day the change shipped. The oldest and most examined records are the ones left bare.	The same bytes on a second export
Two dates	One to two weeks	The rule versions that applied to past choices were never kept, so they cannot be worked out from anything. The system can say what it believes now, which is not the question.	Run any day again, get the same answer
Rules as content	Two to three weeks	Rules already in code have to be pulled out of it and every customer moved. In practice this gets put off, and the layer becomes a set of one off builds.	Ship a duty with no code release
Customers apart at the key	Days, if designed in	Adding it later means re-keying live customer data. It is the most likely place for a breach to be let in, and it would be let in by the fix.	A cross customer read returns nothing

None of these four is dear at the start. All four are close to out of reach at the end, which is the whole case for spending the first release on the layers nobody can see.

16 One morning, step by step

One morning, seven steps, and no person until step six

Read top to bottom. The order is the point, and the clock is a column of its own.

■ **Blue:** the software does it, with nobody there ■ **Green:** a named person signs. Step six only.

1	Feeds Files land	02:00	Statements, the customer ledger and payout files arrive. The raw file is kept exactly as it came.
2	Ingest Files checked	02:10	Names matched, repeats dropped, fields checked. A file that will not read fails loudly rather than quietly.
3	Rule store Rule picked	06:00	Which duty applies, and which version of the rule was in force on that date.
4	The software Money compared	06:05	The gap is worked out, the shortfall test is run, and every unexplained item is listed.
5	Evidence store Result stored	06:06	Result, inputs, rule version and signer written once. No way exists to change any of it later.
6	Screens A person signs	09:14	A named person reads the list of gaps and signs. Stamped with the time, the name and the rule version.
7	Filing Report filled in	31 Mar	The same record becomes a filled in box on the yearly report, with a trail back to this morning.

The software does the work. The person carries the authority. Both are written down as they happen, not put back together afterwards.

Steps one to five run with nobody there. The person shows up once, at step six, to read the list of unexplained items and sign. The system does the work and the person carries the authority, because a sign off is only proof if a person with the right to sign gave it.

Each of those seven steps is written down as it happens. Nothing on this page is put back together afterwards from a log or from memory. The order is fixed, and a step that cannot finish stops the morning rather than guessing its way past. What each step does when it cannot finish is set out on the next page.

17 The failure path for each of the seven steps

STEP	THE FAILURE	WHAT THE SYSTEM DOES
1 Feeds	A file does not turn up, or turns up cut short.	Raises a missing feed alert against the duty, not into a queue. Silence is never taken for success.
2 Ingest	A field will not read, or a name cannot be matched.	Fails the file loudly and holds it. Never fills a blank, never half loads, never guesses a name.
3 Rule store	No duty version covers the day being worked.	Stops, and asks the person who owns the rules. Working under a guessed rule is worse than not working.
4 The core	The gap is over the limit, or the shortfall test fails.	Makes the result anyway, flagged, with the list attached. A failing check is still a record.
5 Evidence store	The write cannot finish.	The whole step fails and runs again. There is no half write, because half a record breaks the chain.
6 Screens	Nobody signs by the cut off.	The duty goes late on the Today screen and is escalated. The record shows unsigned, which is itself the proof.
7 Filing	The rule maker's portal turns the payload down.	Keeps the refusal as a record, tries again on a clock, and never quietly reshapes the payload to make it pass.

Every row is a choice about what the system does when it does not know. The honest answer is nearly always to stop and say so, because a made up carry on cannot be explained later.

WHAT EACH TERM ON THIS PAGE MEANS

Feed. An automatic daily file from a bank or a custodian, arriving without being asked for.

Cut off. The time of day by which the day's check has to be signed.

Gap. The difference between the two sides of a check. Zero is the expected answer.

Escalate. Push it up to a named person with the standing to act on it.

Limit. The size of gap above which the system raises an alert rather than noting it.

Portal. The rule maker's own website for taking filings in.

18 Keeping clocks, run by the system rather than written as a policy

DUTY	KEPT FOR	WHAT THE CLOCK HAS TO DO
Money records	As the rule says, from the date of the record	Start on the record date, not the date it was loaded, because a back dated fix does not restart the span it fixes.
Plan and reviews	At least through the next outside review	Live through a rule change. A replaced plan version is still the version that applied when it applied.
Open banking proof	Five years, under rules drafted 27 June 2026	Keep sign off paperwork, consent records and incident write ups on a clock of their own.
Read events	As long as the records they touch	A read log that runs out before its records is worse than no log, because it implies a fullness it does not have.

Keeping clocks is where rule software usually fails quietly. Deleting on time is easy. Proving that nothing went early, and that what went was meant to go, is the part that needs the chain. Every clock above starts on a date the record itself carries.

19 The eleventh job, shipped without a builder

A new duty is a piece of writing, not a piece of code

If any part of adding one needs a code release, the second rule maker costs a second build.

Written, not coded

signed by the rules owner

DUTY

What it is, and its dates

FIELDS

The boxes to fill in

STEPS

Who does what, and when

LINKS

Proof matched to duty

The core

built once, not per duty

CHECK

Two sets of figures that must agree

PAPER

A written thing with a signer

FILING

A closed form, checked then sent

Made

written once, never changed

PROOF RECORD

With rule version and signer

REPORT BOX

With a trail back to it

EXAM PACK

Indexed, with who read what

REVIEWER EXPORT

The same bytes a year later

How it must not work

New duty ► new fields ► a ticket ► a code review ► a release ► eleven products in one coat

How it does work

New duty ► content signed by the rules owner ► live ► the second rule maker costs a setting

Fourteen duties, four rule makers, three shapes. A new job is a set of fields, a set of steps and a set of links, written by the person who owns the rules and released as content. If any part of that needs a code release, the job is a product rather than a setting, and the money behind everything downstream changes with it.

EVERY ENGINEERING WORD ON THIS DRAWING, AND WHAT EACH ONE MEANS

Duty. One thing a rule maker requires, with the dates it applies between.

Release. A dated, signed change to the rule store. It ships without a code change.

Shape. One of the three forms every duty comes down to: a check, a paper with a signer, or a filing.

Fields. The named boxes a duty asks to be filled in.

Content. Writing held as data, which the core reads and runs. It is not code.

Links. The written tie between a duty and the proof that answers it.

The test is easy to run and hard to pass. Take the fourteenth duty on the list, hand it to the person who owns the rules, and watch whether it goes live without a builder. If it does, the tenth customer costs no more to serve than the first. If it does not, every rule maker after the first is a second product.

20 Worked example. The second rule maker, written entirely as content

job: property-trust-monthly

```
# Alberta Real Estate Act ss.25, 69;  
Rules 41, 52, 91, 95 to 97  
shape: check  
how often: monthly  
due: 30d after last statement date  
sides: [client_records, trust_account]  
needs:  
  - held_per_client # one line each  
  - client_record_note  
signer: broker_or_delegate  
split_duties: preparer != signer  
keep_for: as the rule says  
links_proof: [check_result,  
held_per_client, sign_off]
```

What the build team has to write to add it

- Nothing. The check shape is already there.
- How often, when it is due and who signs are fields, not branches in code.
- The list of what is held per client is linked to the duty rather than added as a table.
- Splitting the work from the sign off is a rule the core already keeps.
- This is the test of the whole idea, and it is worth running on purpose rather than waiting for a customer to run it for us.

WHAT EACH TERM ON THIS PAGE MEANS

Job. One whole duty the system runs end to end, written as content rather than code.

Signer. The job the signing person has to hold before a sign off counts.

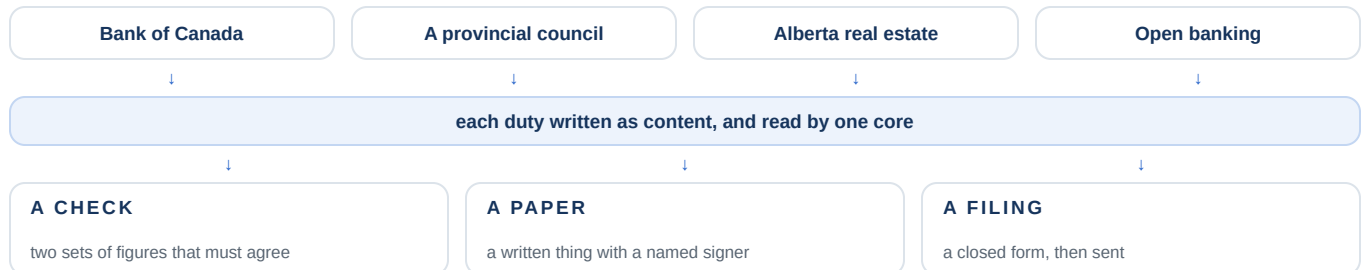
Shape. One of the three forms every duty comes down to: a check, a paper with a signer, or a filing.

Split duties. The rule that whoever does the work cannot be whoever signs it off.

How often. When the duty falls due. Daily, monthly, yearly, or on an event.

Link. The written tie between a duty and the proof that answers it, readable both ways.

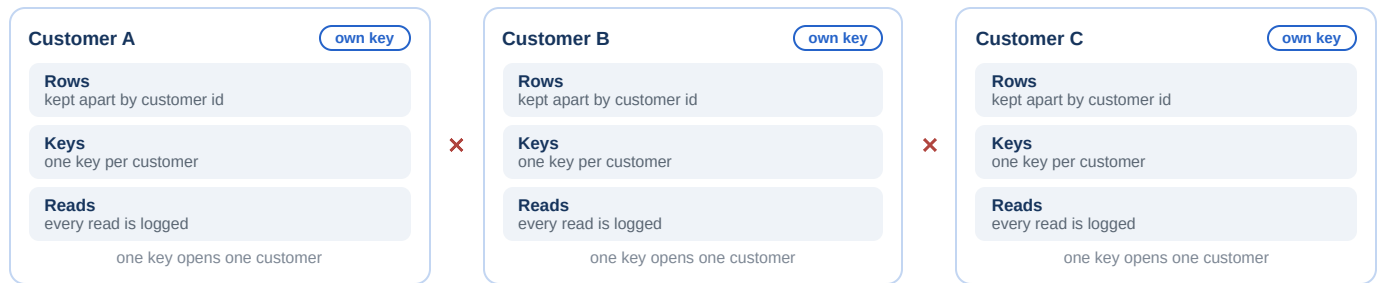
Four rule makers, and the same three shapes under all of them



Not one of the four needs a shape the core does not already have. The duty above is Alberta property trust money. It is a check, on a monthly clock, with a signer who cannot also be the preparer. Every one of those is a field.

21 Keeping customers apart, and where the data lives

Canada, the region named in the contract



One core underneath, and it does not know what a payment firm is

It knows about checks, papers with signers, and filings. Customers are kept apart below it, at the row and at the key, so the mistake cannot be made by a bad query.

Reads by us are logged the same way reads by the customer are

There is no standing login to live data, for anybody, the founder included.

Keeping customers apart belongs below the core, not inside the query. Kept at the row and at the key, the mistake is not there to be made. Engineers call the first of those tenant isolation, which means one customer's data can never be reached from another customer's, and the second data residency, which means the country the data sits in.

EVERY ENGINEERING WORD ON THIS DRAWING, AND WHAT EACH ONE MEANS

Customers kept apart. One customer's data can never be reached from another customer's. Engineers call this tenant isolation.

At the key. One key for each customer, so one key opens one customer and no more.

Where the data sits. The country the records are held in. Ours is Canada, and it is written into the contract. Engineers call this data residency.

Read event. A record that somebody looked at something, kept on the same terms as the thing they looked at.

At the row. Every record carries the customer it belongs to, and that is checked below the query rather than inside it.

Standing login. An account that is always live and always able to read customer data. There is none here, for anybody.

22

The eight security review questions, and the answer each one needs

THE QUESTION A WATCHED BUYER ASKS	THE ANSWER THAT HAS TO ALREADY BE TRUE	LAYER
Where does our data live?	Canada, written into the contract, with the region named and the list of suppliers published.	Platform
Can your staff see our records?	Only with a logged read attached. Reads by us are logged the way reads by the customer are, and nobody holds a standing login to live data.	Platform
Can a record be changed after the fact?	There is no way to change and no way to delete in the code. Fixes replace rather than write over, and the replaced record stays readable.	Evidence store
Prove that to our outside reviewer.	Two exports of the same month taken a year apart, compared byte for byte.	Evidence store
What happens when the rule changes?	A signed release by the person who owns the rules, with the past run again and the difference shown rather than quietly restated.	Rule store
Does your assistant read our data?	One customer at a time, never across customers, each answer tied to a record id, and every use logged as an event.	Assistant
Who are your subcontractors?	Named in the agreement and disclosed, with notice before any change. This has been a standing rule since 1 May 2024.	Platform
What if you are bought, or fail?	Export in every view, in a form the customer can read without us, and a copy of the rule store content lodged with a third party.	Screens

Eight questions, and each one has to be answerable in the first sales call rather than in the third. A watched buyer treats a pause here as a finding.

WHAT EACH TERM ON THIS PAGE MEANS

- Where data lives.** The country the data sits in. Ours is Canada, and it is in the contract.

Lodged with a third party. A copy held by a neutral firm and handed to the customer if we cannot carry on. Lawyers call this escrow.
- Supplier.** Another firm we use to run the service, who therefore touches customer data.

Key. The secret that unlocks stored data. One key per customer means one key opens one customer.
- Read event.** A record that somebody looked at something, kept on the same terms as the thing they looked at.

Standing login. An account that is always live and always able to read customer data. There is none.

Every answer above is a thing already built, not a thing promised.

A watched buyer can ask for the proof of any one of the eight in the first call. That is the whole reason the layers nobody can see are built first.

The eight answers are not a security policy. Each one names a layer on page four, and each one can be shown running rather than described. A reviewer who is handed a document rather than a running system treats the pause as a finding. The eight come up in roughly this order, and a pause on any one of them costs the same as a wrong answer.

23 The exam pack, put together rather than rebuilt

Four minutes, not three weeks

Building a pack is itself an event with a time, a scope and a name against it, so answering an examiner is recorded like everything else.



The rule as it stood on each day is the part almost nothing else can produce. A duty done in March 2026 was done under the rule as it stood in March 2026, and the pack has to say so without being asked.

The pack is itself a record. Making one is an event with a time, a scope and a name against it, so answering an examiner is written down on the same terms as everything else in the system. The pack is built when it is asked for rather than kept ready, because a pack kept ready goes stale the day after it is made.

24 The contents of a filing pack, the piece most often under counted

The pack, one per rule maker per year

- The map from our record shape to the rule maker's form, written as content rather than as code.
- The checks, run before sending and never after, with the failure shown against the duty.
- The way it is sent, whether a portal or an upload, and how we prove who we are.
- The receipt, the retry rule and the failure handling, because a filing that fails at 23:50 on 31 March is the case that matters.

And what it has to keep being able to do

- Draw last year's filing exactly as it was sent, under last year's form, three years from now.
- Show the trail from each figure back to the record that made it, in one click.
- Prove when it was sent without leaning on our own clock.
- Live through a mid year change to the form without spoiling anything already sent.
- Take a refusal from the portal, keep it as a record, and try again on a clock.

A filing pack gets counted as a small piece of work because the form is short. The form is the easy half. The hard half is drawing last year exactly as it was sent, three years from now, under a form that has since changed twice.

25 The split of the build, and the layers not handed over

1 Ours, and not handed over

- The evidence store and the two dates. Both carry the whole position.
- The filing packs, because each rule maker's form is fixed and closed.
- The rule store content itself.

2 Speer, to our design

- Platform: keeping customers apart, where data lives, keys, logins, recovery.
- Ingest, screens and the core, built to the shapes we set.
- At the hourly rates set out below.

Why the split holds

- Each layer that carries the position stays in house, so a change of partner is a delay.
- One month written notice, either way.
- Everything built belongs to 4orm Finance the day it is made.

If the eleventh job needs code, the whole idea has failed.

That one sentence is the test behind each choice in this pack. It is why the five screens are fixed and why the rules are held as content.

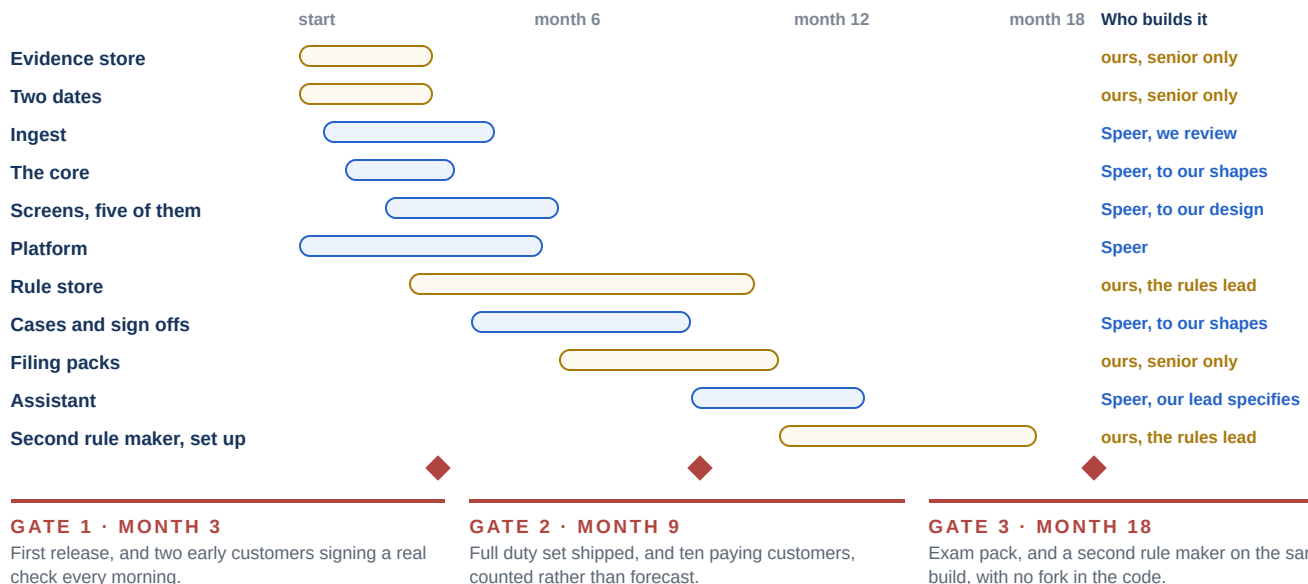
26 The team, the rates, and the money behind each layer

ROLE	RATE	OWNS	#	WHEN	AMOUNT	WHAT IT PAYS TO BUILD
Senior build, ours	To hire	Evidence store, two dates, filing packs. Not handed over.	1	Aug 2026	\$50,000	Legal ground. The company, counsel, and the paper that puts each layer into 4orm Finance the day it is made.
Rules lead, ours	Part time	The rule store content, and the signature on each release.	2	Aug 2026	\$500,000	Evidence store and two dates, started by senior build on day one. Platform and ingest start beside them.
Build manager	\$150 an hour	The core, the reader of written jobs, and saying no to shape changes.	3	Oct-Nov 2026	\$500,000	The core and the five screens, with the rule store opened. First release, which is Safeguard live.
Full stack	\$130 an hour	Ingest, screens, feeds, and the run again path.	4	Feb-Mar 2027	\$500,000	Cases and sign offs, and the filing packs behind the 31 March report. This is Comply.
Senior design	\$140 an hour	Five screens and no more, and export in every view.	5	Apr-May 2027	\$500,000	Assistant, exam pack, and the full duty set running on one core.
Platform	\$100 an hour	Platform, customers apart, keys, region, recovery, backups.	6	Aug 2026	\$500,000	No layer. It buys time rather than code, and it is switched off in every figure in this pack.

Rates are Canadian dollars an hour, agreed in advance and invoiced against a named piece of work, so each hour maps to something on the plan on page twenty. The build work can be stopped by either side on one month written notice without penalty.

Who builds what, and when

The two hardest layers start on day one and finish first. That is the opposite of the usual order.



■ ours, never handed over
 ■ Speer, our build partner
 ■ a gate, and the money it releases

The two hardest layers start on day one and finish first. That is the opposite of the usual order, in which the screens ship first and the parts nobody can see are promised for a release that later gets squeezed.

The build work can be stopped by either side on one month written notice without penalty, which caps the exposure at each step. Three of the five parts of the raise are released against something being true rather than against a date. All amounts Canadian dollars.

Read the plan by what finishes first rather than by what starts first. The evidence store and the two dates are the two layers a customer never sees, and the two that cannot be added afterwards. Both are done before the first screen is drawn. The screens are held to a fixed set of five, so the usual squeeze at the end has nothing left to take.

28 Twelve questions for the architect, in the order they matter

The first six are about the record and the store. The second six are about rules, failure and proof. A good answer names a thing, not a wish, and it says where that thing lives. The last question on the list matters most, because a pack nobody argues with has not been read closely.

- 1 **Where does add only actually live?** In the store, in the app, or in a habit? Name the thing, not the wish.
- 2 **How would you hold the two dates?** Two date pairs on the row, a history table, or a built in feature? Each has a different cost to run a day again.
- 3 **What does a fix look like?** Show the write path for a value fixed six weeks later, and what the first reader sees afterwards.
- 4 **How is the chain anchored?** Per customer, per month, or one for all? And what stops a chain being rebuilt whole?
- 5 **Where are customers kept apart?** If the answer has the word query in it, we should talk before anything is written.
- 6 **How are keys split per customer?** Wrapped keys, a key tree, and what happens when a key is swapped.
- 7 **What reads a job written as content?** The thing that turns writing into a running duty is the least showy part here and the one that decides the whole idea.
- 8 **How do you run history again after a rule change?** Run again and show the difference, or run again and restate? Only the first is acceptable.
- 9 **What does the ingest failure path look like?** Say what happens to a file that half reads.
- 10 **How do you test that nothing can be changed?** A line in a document is not a test. What runs on every code change?
- 11 **What is the export form?** It has to be readable without us, and the same bytes when exported again a year later.
- 12 **What would you push back on in this pack?** The most useful answer on the list. If nothing here is worth arguing with, it has not been read closely.

Build the two hardest things first, because they are the two that cannot be added later.

Invisible on a demo, and the whole reason a watched buyer would trust us with the record it will be examined on.

EVERY ENGINEERING WORD IN THESE QUESTIONS, AND WHAT EACH ONE MEANS

Add only. New records can go in. Nothing already written can be changed or removed. Engineers call this append only.

Run history again. Work a past day out afresh under the rule that applied on that day.

Anchored. Tied to a fixed point so a whole run of records cannot be rebuilt quietly.

Export form. The shape the records are handed over in. It has to open without us.

Chain. Each record carries the fingerprint of the one before it, so altering one shows up in plain sight.

Key tree. Keys held under other keys, so one customer's key can be swapped on its own.